⚙

GOLF FUNDRAISER PRO

Foundation Document

Foundation & Shared Infrastructure

Monorepo · Database Schema · Auth · Theme · DevOps — read before any phase

Golf Scramble Made Easy · Implementation Design Series · v1.0

1. How to Use This Document Series

Golf Fundraiser Pro is built in six phases. Each phase has its own implementation design document. This Foundation document covers everything that is shared across all phases — the monorepo structure, the complete database schema, the authentication system, the theme engine, and the DevOps pipeline. Read this document once before starting any phase. Do not duplicate its content in phase-specific documents; reference it instead.

> **Token Efficiency:** When working with an AI assistant on a specific phase, load this Foundation document plus that phase's document. You do not need to load all six phase documents simultaneously. The Foundation + one Phase document is sufficient context for implementing that phase.

Total Phases	6 phases + this Foundation
Primary Language	C# (.NET 8) — API; TypeScript — all frontend
Mobile	React Native + Expo SDK 55 (New Architecture mandatory)
Admin Web	Expo Router web (shared components with mobile)
Public Web	Next.js 15 App Router (SSR for SEO)
Database	PostgreSQL 16 + PostGIS 3.4
Container Runtime	Docker Compose (local dev); Railway/Render (production)

2. Monorepo Structure

The entire project lives in a single Turborepo monorepo. All apps share the packages/ui component library, packages/shared-types TypeScript DTOs, and packages/theme color tokens.

```
golf-fundraiser-pro/
  apps/
    api/          # ASP.NET Core Web API (.NET 8)
    admin/        # Expo Router web — organizer dashboard
    mobile/       # Expo SDK 55 — iOS & Android scoring app (Phase 2+)
    web/          # Next.js 15 — public leaderboard, landing page (Phase 2+)
  packages/
    ui/           # Shared React Native components (mobile + admin)
    shared-types/ # TypeScript DTOs — API request/response contracts
    theme/        # 5-token color system, WCAG validation, org overrides
  infra/
    docker-compose.yml  # Postgres/PostGIS, Redis, pgAdmin, API
    nginx.conf          # Local reverse proxy
  .github/
    workflows/    # CI: lint, type-check, test, build all apps
  turbo.json      # Pipeline: build depends on ^build, lint is parallel
package.json    # Root workspace with shared dev dependencies
```

New Architecture: React Native New Architecture (Fabric + TurboModules) is mandatory from day one. Do not use the legacy bridge. Expo SDK 55 enables New Architecture by default — do not disable it.

3. Technology Stack — Complete Reference

Layer	Technology	First Used	Key Notes
Admin Dashboard	Expo Router web	**Phase 1**	Shares packages/ui with mobile. EAS Hosting. No SEO needed — behind auth.
Public Leaderboard	Next.js 15 App Router	**Phase 1**	SSR for SEO + cold-load speed. Vercel. TV mode. JSON-LD Event structured data.
Mobile App	React Native + Expo SDK 55	**Phase 2**	New Architecture mandatory. EAS Build. expo-sqlite v14 for offline.
Backend API	ASP.NET Core Web API (.NET 8)	**Phase 1**	C# 12. Minimal API style for endpoints. Entity Framework Core for ORM.
Database	PostgreSQL 16 + PostGIS 3.4	**Phase 1**	Full schema created in Phase 1 migrations. PostGIS for GPS in Phase 6.
Offline DB	SQLite (expo-sqlite v14)	**Phase 2**	WAL mode. pending_scores queue. Full event cache pre-downloaded at join.
Real-Time	ASP.NET SignalR	**Phase 3**	TournamentHub. Score + auction events. Redis backplane for multi-instance.
Cache	Redis 7	**Phase 3**	Leaderboard TTL cache (2–5s). StackExchange.Redis. Docker in dev.
Payments	Stripe	**Phase 4**	Setup Intents (card-on-file), PaymentIntents, Webhooks. No markup on pass-through.
Email	SendGrid	**Phase 1**	.NET SDK. 100 emails/day free tier. All templates in DB, org-branded.
Push Notifications	Expo Notifications	**Phase 3**	iOS APNs + Android FCM via Expo push token. Outbid + Hole-in-One alerts.
Auth	ASP.NET Identity + JWT	**Phase 1**	15-min access token. Refresh in SecureStore (mobile) / httpOnly cookie (web).

Maps	Mapbox GL	**Phase 6**	@rnmapbox/maps + mapbox-gl-js. 50k free loads/month. Offline tile cache.
Monorepo Build	Turborepo + GitHub Actions	**Phase 1**	Parallel lint, type-check, test. EAS Build for mobile. Vercel for Next.js.

4. Complete Database Schema

The full schema is created in Phase 1 migrations. All tables exist from day one — later phases write to columns that were always there. This prevents breaking migrations on live data. Columns populated in later phases are nullable until that phase is deployed.

Migration Strategy: Use EF Core migrations with a single DbContext. Each phase adds a named migration: Phase1_InitialSchema, Phase2_AddOfflineColumns, Phase4_AddAuction, Phase5_AddLeague, Phase6_AddGPS. Never drop or rename existing columns — add new ones.

4.1 Core Tables (Phase 1)

Column / Table	Type	Description
organizations	Table	id, name, slug, logo_url, theme JSONB, mission_statement, is_501c3, created_at
id	UUID PK	All tables use UUID primary keys generated by the API (not DB serial)
slug	VARCHAR(60) UNIQUE	URL-safe org identifier, e.g. 'clhs'. Used in /e/{slug}/{eventCode} routes.
theme	JSONB	{ primary, action, accent, highlight, surface } — 5-token color override. NULL = use Eco Green defaults.
events	Table	id, org_id FK, name, event_code CHAR(8) UNIQUE, format ENUM, start_type ENUM, holes SMALLINT, status ENUM, start_at TIMESTAMPTZ, config JSONB
event_code	CHAR(8) UNIQUE	Random alphanumeric. Used in QR codes and public URLs. Indexed.
format	ENUM	scramble | stroke | stableford | best_ball | match
start_type	ENUM	shotgun | tee_times
status	ENUM	draft | registration | active | scoring | completed | cancelled
config	JSONB	Flexible event settings: allow_walk_ups, max_teams, tee_intervals, free_agent_enabled, etc.
courses	Table	id, org_id FK, name, address, city, state, zip, location GEOGRAPHY(Point,4326)

course_holes	Table	id, course_id FK, hole_number SMALLINT, par SMALLINT, handicap_index SMALLINT, yardage_white, yardage_blue, yardage_red, cup_location GEOGRAPHY(PointZ,4326) NULL←Phase 6, cup_elevation_m REAL NULL←Phase 6
handicap_index	SMALLINT	Hole difficulty rank 1–18 (1=hardest). Used to allocate net strokes in match play and league net scoring.
teams	Table	id, event_id FK, name, captain_player_id FK, starting_hole SMALLINT, tee_time TIMESTAMPTZ, entry_fee_paid BOOL, invite_token VARCHAR(64) NULL, invite_expires_at TIMESTAMPTZ NULL, max_players SMALLINT DEFAULT 4, check_in_status ENUM
players	Table	id, team_id FK NULL, event_id FK, first_name, last_name, email UNIQUE per event, phone, handicap_index REAL NULL, registration_type ENUM, skill_level ENUM NULL, age_group ENUM NULL, pairing_note TEXT NULL, check_in_status ENUM, check_in_at TIMESTAMPTZ NULL, has_payment_method BOOL DEFAULT false←Phase 4, stripe_customer_id VARCHAR NULL←Phase 4
registration_type	ENUM	full_team | individual_join | free_agent | free_agent_assigned | walk_up
scores	Table	id, event_id FK, team_id FK, hole_number SMALLINT, gross_score SMALLINT, putts SMALLINT NULL, player_shots JSONB NULL, device_id VARCHAR, submitted_at TIMESTAMPTZ, synced_at TIMESTAMPTZ NULL, source ENUM, is_conflicted BOOL DEFAULT false, drive_location GEOGRAPHY(Point,4326) NULL←Phase 6, ball_location GEOGRAPHY(Point,4326) NULL←Phase 6
source	ENUM	admin_entry | mobile_sync | qr_transfer
hole_challenges	Table	id, event_id FK, hole_number SMALLINT NULL, challenge_type ENUM, description TEXT, prize_description TEXT NULL, sponsor_id FK NULL
challenge_type	ENUM	closest_to_pin | longest_drive | putting | beat_the_pro | hole_in_one | straightest_drive
challenge_results	Table	id, challenge_id FK, team_id FK, player_id FK NULL, result_value REAL NULL, result_notes TEXT NULL, recorded_at TIMESTAMPTZ
sponsors	Table	id, event_id FK, name, logo_url, website_url NULL, tagline NULL, tier ENUM, placements JSONB
tier	ENUM	title | gold | hole | silver | bronze
donations	Table	id, event_id FK, donor_name, donor_email, amount_cents INT, receipt_sent BOOL, stripe_payment_intent_id VARCHAR NULL←Phase 4, created_at TIMESTAMPTZ
email_templates	Table	id, org_id FK, trigger_type ENUM, subject, html_body, is_active BOOL

trigger_type	ENUM	registration_confirm | team_assignment | invite | reminder | thank_you | donation_receipt | round_result←Phase 5 | handicap_update←Phase 5
qr_codes	Table	id, event_id FK, qr_type ENUM, token VARCHAR(128) SIGNED, svg_data TEXT, created_at TIMESTAMPTZ
qr_type	ENUM	event_join | player_checkin | leaderboard | donation | hole_challenge | app_download

4.2 Phase 4 Additions — Payments & Auction

Column / Table	Type	Description
stripe_customers	Table	id, player_id FK UNIQUE, stripe_customer_id VARCHAR UNIQUE, default_payment_method_id VARCHAR NULL, created_at
auction_items	Table	id, event_id FK, title, description TEXT, photos VARCHAR[], fmv_cents INT, min_bid_cents INT, bid_increment_cents INT, buy_now_cents INT NULL, auction_type ENUM, donation_denominations INT[] NULL, opens_at TIMESTAMPTZ, closes_at TIMESTAMPTZ, original_closes_at TIMESTAMPTZ, max_extension_minutes SMALLINT DEFAULT 10, donor_name NULL, category NULL, is_featured BOOL, status ENUM
auction_type	ENUM	silent | live | donation_silent | donation_live
status (item)	ENUM	draft | open | extended | closed | awarded | cancelled
bids	Table	id, item_id FK, player_id FK, amount_cents INT, placed_at TIMESTAMPTZ, is_buy_now BOOL DEFAULT false, stripe_pm_id VARCHAR NULL. UNIQUE(item_id, player_id) for donation items uses a different constraint.
auction_winners	Table	id, item_id FK, player_id FK, winning_amount_cents INT, awarded_at TIMESTAMPTZ, stripe_payment_intent_id VARCHAR NULL, charge_status ENUM, receipt_sent BOOL, pickup_notes TEXT NULL
charge_status	ENUM	pending | succeeded | failed | waived
auction_sessions	Table	id, event_id FK, session_type ENUM, started_at TIMESTAMPTZ, ended_at TIMESTAMPTZ NULL, current_item_id FK NULL

4.3 Phase 5 Additions — League Play

Column / Table	Type	Description
leagues	Table	id, org_id FK, name, season_year SMALLINT, format ENUM, handicap_system ENUM, max_flights SMALLINT DEFAULT 1, dues_cents INT DEFAULT 0, status ENUM
seasons	Table	id, league_id FK, name, start_date DATE, end_date DATE, total_rounds SMALLINT, status ENUM

flights	Table	id, season_id FK, name, min_handicap REAL NULL, max_handicap REAL NULL, max_players SMALLINT NULL
league_members	Table	id, season_id FK, player_id FK, handicap_index REAL, flight_id FK NULL, dues_paid BOOL DEFAULT false, rounds_played SMALLINT DEFAULT 0, absences SMALLINT DEFAULT 0, active BOOL DEFAULT true
league_rounds	Table	id, season_id FK, round_number SMALLINT, round_date DATE, course_id FK, format_override ENUM NULL, status ENUM, tee_assignment JSONB NULL
league_pairings	Table	id, round_id FK, group_number SMALLINT, player_ids JSONB, tee_time TIMESTAMPTZ NULL, starting_hole SMALLINT NULL, is_sub BOOL DEFAULT false
league_scores	Table	id, round_id FK, player_id FK, hole_number SMALLINT, gross_score SMALLINT, net_score REAL NULL, stableford_points SMALLINT NULL, match_result CHAR(1) NULL, putts SMALLINT NULL
match_result	CHAR(1)	W=Win, L=Loss, H=Halve — for match play format
handicap_history	Table	id, member_id FK, old_index REAL, new_index REAL, round_id FK NULL, calculation_method VARCHAR, calculated_at TIMESTAMPTZ, admin_override BOOL DEFAULT false, override_reason TEXT NULL
standings	Table	id, season_id FK, player_id FK, flight_id FK NULL, total_points REAL, wins SMALLINT, losses SMALLINT, halves SMALLINT, avg_gross REAL, avg_net REAL, rounds_counted SMALLINT, last_updated TIMESTAMPTZ. Refreshed after each round closes.
skins	Table	id, round_id FK, hole_number SMALLINT, winner_player_id FK NULL, pot_cents INT, carried_over_from_hole SMALLINT NULL

5. Authentication & Authorization

5.1 JWT Flow

All protected API endpoints require a Bearer JWT. The access token is short-lived (15 minutes). The refresh token is long-lived (30 days, rotating).

- POST /auth/register — create organizer account (email + password). Returns access + refresh tokens.
- POST /auth/login — validate credentials. Returns access + refresh tokens.
- POST /auth/refresh — exchange refresh token for new access + refresh pair. Old refresh token invalidated immediately.
- POST /auth/logout — invalidate refresh token server-side.

5.2 Token Storage by Platform

Platform	Access Token	Refresh Token

Mobile (React Native)	In-memory (Zustand store). Never persisted to disk.	expo-secure-store. Encrypted, hardware-backed on iOS.
Admin web (Expo Router)	In-memory. React context.	httpOnly cookie. Sent automatically by browser.
Public web (Next.js)	Not required — leaderboard is public. No auth.	n/a

5.3 Role System

- OrgAdmin — full access to their organization's events, members, and settings
- EventStaff — check-in and score entry for a specific event; no settings access
- Golfer — score entry for their own team; read leaderboard; bid in auction
- Public — read-only leaderboard and event landing page; no auth required
- Role is stored in the JWT claims. API endpoints use [Authorize(Roles = "OrgAdmin")] attributes.

6. Theme & Color System

Every organization gets a 5-token color palette applied across all surfaces: admin dashboard, mobile app, public leaderboard, emails, and PDFs.

6.1 Default Eco Green Palette

Token	Default Hex	Semantic Name	Usage
--color-primary	#31572c	Dark Forest	Nav, header, footer, primary button fill
--color-action	#409151	Leaf Green	CTAs, links, active tabs, leaderboard highlights
--color-accent	#8ba955	Sage	Hover states, secondary badges, sponsor strip bg
--color-highlight	#ecf39e	Pale Lime	Selected states, callout banners, QR borders
--color-surface	#f4f7de	Cream	Page backgrounds, card surfaces, email body bg

WCAG Compliance: Dark Forest (#31572c) on Cream (#f4f7de) = 7.4:1 (AAA). Leaf Green on Cream = 4.6:1 (AA). All default pairings meet WCAG 2.1 AA minimum. Contrast validation blocks theme save if primary text pairing fails.

6.2 Token Distribution by Platform

- Next.js: CSS custom properties injected in root layout.tsx at SSR time — zero flash of unstyled content.
- Expo Router web: same CSS custom properties via shared theme package.
- React Native mobile: ThemeContext provides tokens via useTheme() hook to all components.

- Email: inline style substitution at SendGrid render time — {{PRIMARY_COLOR}} replaced with org hex.
- Per-event override: events.config.theme_override JSONB (nullable) — null inherits org theme.

7. Shared Packages

7.1 packages/shared-types

TypeScript interfaces and Zod schemas for every API request and response. Imported by both the API (for request validation) and all frontend apps (for type-safe data fetching). Never define the same DTO shape in two places.

- EventDTO, TeamDTO, PlayerDTO, ScoreDTO, LeaderboardDTO
- AuctionItemDTO, BidDTO, AuctionWinnerDTO
- LeagueDTO, SeasonDTO, LeagueMemberDTO, StandingsDTO
- All enums re-exported from here so they're shared across API and frontend

7.2 packages/ui

React Native components shared between the mobile app (apps/mobile) and the admin dashboard (apps/admin). Components must work in both React Native and React Native Web (Expo Router web output).

- ScoreCard — per-hole gross score entry with ± buttons and 56pt touch targets
- LeaderboardRow — team name, to-par, thru, sponsor badge
- TeamCard — player roster, starting hole, tee time, check-in status
- SponsorBanner — logo + tier badge, used in scoring screens and leaderboard
- ThemeProvider — wraps app root; provides useTheme() hook
- Rule: no platform-specific code (no Platform.OS checks) in packages/ui. Use StyleSheet.create for all styles.

7.3 packages/theme

- ECO_GREEN_DEFAULT — the 5-token default palette as a typed object
- validateContrast(primary, surface) → boolean — WCAG 4.5:1 check using relative luminance formula
- buildCSSVars(theme) → string — generates :root { --color-primary: #... } block for web
- buildThemeContext(theme) → ThemeContextValue — object for React Native ThemeContext

8. Docker Compose — Local Development

All backend services run in Docker Compose. The API runs locally via dotnet run (hot reload) and connects to the containerized database and Redis.

```
services:
  postgres:
    image: postgis/postgis:16-3.4
    ports: ["5432:5432"]
    environment:
      POSTGRES_DB: golf_fundraiser
      POSTGRES_USER: gfp
      POSTGRES_PASSWORD: gfp_local
```

```
    volumes: [pgdata:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  pgadmin:
    image: dpage/pgadmin4
    ports: ["5050:80"]
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@local.dev
      PGADMIN_DEFAULT_PASSWORD: admin

volumes:  pgdata:
```

PostGIS: Always use the postgis/postgis image, not the plain postgres image. The PostGIS extension must be created with CREATE EXTENSION postgis; in the initial EF Core migration's raw SQL block.

9. CI/CD Pipeline

9.1 GitHub Actions Workflows

- ci.yml — runs on every PR: turbo lint, turbo type-check, turbo test. All must pass before merge.
- deploy-api.yml — runs on merge to main: docker build API image → push to registry → Railway/Render deploy.
- deploy-web.yml — runs on merge to main: vercel deploy --prod (Next.js apps/web).
- deploy-admin.yml — runs on merge to main: eas deploy --prod (Expo Router web apps/admin).
- eas-build.yml — triggered manually or on tag: eas build --platform all (iOS + Android).

9.2 Environment Variables

Never commit secrets. All environments use the following variables, stored in GitHub Secrets for CI and in Railway/Render environment config for production.

Variable	Description
DATABASE_URL	PostgreSQL connection string. Format: postgresql://user:pass@host:5432/db?sslmode=require
REDIS_URL	Redis connection string. Format: redis://host:6379
JWT_SECRET	Random 64-byte hex string for JWT signing
JWT_REFRESH_SECRET	Separate secret for refresh tokens
SENDGRID_API_KEY	SendGrid API key. Free tier: 100 emails/day
STRIPE_SECRET_KEY	Stripe secret key (Phase 4+). Never expose client-side.
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret for /webhooks/stripe endpoint
MAPBOX_TOKEN	Mapbox public token (Phase 6+). Safe to expose client-side.

NEXT_PUBLIC_API_URL	Public API base URL for Next.js server-side and client-side fetching
EXPO_PUBLIC_API_URL	Public API base URL for Expo apps (EXPO_PUBLIC_ prefix required)

10. API Conventions

All API endpoints follow these conventions. Phase documents define specific endpoints; they all comply with these rules.

- Base URL: /api/v1/...
- Authentication: Authorization: Bearer {accessToken} header on all protected routes
- Request bodies: JSON with Content-Type: application/json
- Pagination: cursor-based where needed. ?after={cursor}&limit={n} (default 50, max 200)
- Timestamps: ISO 8601 UTC strings in all request/response bodies
- Errors: { error: string, code: string, details?: object } with appropriate HTTP status
- IDs: UUID strings (not integers) in all request/response bodies
- Soft deletes: records are never deleted from the database. Use status = cancelled or active = false.

10.1 Standard Error Codes

code	HTTP	Meaning
VALIDATION_ERROR	400	Request body failed validation. details contains field-level errors.
UNAUTHORIZED	401	Missing or expired access token.
FORBIDDEN	403	Authenticated but lacking required role.
NOT_FOUND	404	Resource does not exist or is not visible to this org.
CONFLICT	409	Duplicate unique field (e.g. event_code already exists).
BID_TOO_LOW	422	Bid amount below current high bid + increment.
AUCTION_CLOSED	422	Bid submitted after auction close time.
NO_PAYMENT_METHOD	422	Bid attempted without card on file and not checked in.
HANDICAP_LOCKED	422	Round is closed; handicap recalculation already run.